

NDOT: Neuronal Dynamics-based Online Training for Spiking Neural Networks

论文详细汇总

I. 摘要

本文基于原文 'Algorithm/NDOT/9417_NDOT_Neuronal_Dynamics_ba.pdf' 提取并整理，详细介绍了 NDOT (Neuronal Dynamics-based Online Training) 方法的核心思想、关键公式、算法流程和优势。NDOT 通过神经元动力学表示连续时间依赖，实现前向时间在线训练，并避免传统 BPTT 的高内存开销。本文还总结了 NDOT 在实验中的表现和结论。 [1]

II. 研究背景

脉冲神经网络 (SNN) 通过离散脉冲传播信息，具备能效与时序敏感优势，但其二值脉冲生成函数不可导，使得直接训练面临困难。目前典型方法包括：

- ANN-to-SNN 转换：先训练 ANN 再映射到 SNN，但通常需要更多时间步来维持精度；
- BPTT + surrogate gradient：将 SNN 视为二值 RNN，沿时间展开计算梯度，内存消耗随时间步线性增长；
- 脉冲表示方法：用平均发放率等表征近似时间依赖，降低时序计算但可能增加延迟。

在此背景下，NDOT 提出了一种面向在线学习的前向时间训练框架，直接从神经元动力学中提取连续时间依赖。

III. NDOT 方法

A. 模型与梯度分解

NDOT 的核心思想是将全梯度分解为空间梯度和时间梯度两部分。在经典 BPTT 中，权重梯度具有如下时间展开形式：

$$\frac{\partial L}{\partial W^l} = \sum_{t=1}^T \frac{\partial L}{\partial s^l[t]} \frac{\partial s^l[t]}{\partial u^l[t]} \left(\frac{\partial u^l[t]}{\partial W^l} + \sum_{k < t} \prod_{i=k}^{t-1} \varepsilon^l[i] \frac{\partial u^l[k]}{\partial W^l} \right), \quad (1)$$

其中

$$\varepsilon^l[t] = \frac{\partial u^l[t+1]}{\partial u^l[t]} + \frac{\partial u^l[t+1]}{\partial s^l[t]} \frac{\partial s^l[t]}{\partial u^l[t]} \quad (2)$$

表示离散时间依赖。

NDOT 采用神经元动力学的连续时间依赖表示：

$$e^l[t] = \frac{u^l[t] - V_{th} s^l[t]}{u^l[t-1] - V_{th} s^l[t-1]}. \quad (3)$$

这里 $u^l[t]$ 为第 l 层第 t 个时间步的膜电位， $s^l[t] \in \{0, 1\}$ 为脉冲输出。

基于该表示，NDOT 将全梯度写成：

$$\nabla_{W^l} L = \sum_{t=1}^T g_l^u[t] \hat{a}^{l-1}[t]^\top, \quad (4)$$

其中空间梯度定义为：

$$g_l^u[t] = \left(\frac{\partial L}{\partial s^l[t]} \frac{\partial s^l[t]}{\partial u^l[t]} \right)^\top. \quad (5)$$

时间梯度 $\hat{a}^{l-1}[t]$ 则表示前一层脉冲历史的累积影响。

B. 时间梯度的前向跟踪

NDOT 使用以下方式跟踪时间梯度：

$$\hat{a}^{l-1}[t] = s^{l-1}[t] + \sum_{k < t} P_{k,t}^{l-1} \odot s^{l-1}[k], \quad (6)$$

$$P_{k,t}^l = \prod_{i=k}^{t-1} e^l[i] = \frac{u^l[t-1] - V_{th} s^l[t-1]}{u^l[k-1] - V_{th} s^l[k-1]}. \quad (7)$$

其中 $\odot$ 表示逐元素乘积。

该累积表达式可以递归地写成：

$$\hat{a}^{l-1}[t] = e^{l-1}[t-1] \odot \hat{a}^{l-1}[t-1] + s^{l-1}[t]. \quad (8)$$

这一前向递推关系使得 NDOT 能在时间维上逐步更新，而无需回溯整个历史。

C. NDOT 算法流程

D. 在线损失设计

NDOT 采用即时损失 $L[t]$ ，使每个时间步的梯度可以独立计算，从而支持在线训练。论文强调这与基于平均发放率的离线损失不同，后者需要访问整个时间序列。

IV. NDOT 的优势

leftmargin=*

- **前向时间在线学习：** 梯度计算仅依赖当前时间步和已维护的前一层时间梯度，无需时间维上反向传播；
- **常数训练内存：** NDOT 的状态维护在时间维上为常数，与 BPTT 的 $O(T)$ 内存开销形成对比；
- **连续时间依赖建模：** 通过 $e^l[t]$ 表示神经元的连续动态依赖，保持膜电位和放电后的历史影响；
- **兼容自动微分：** 空间梯度 $g_l^u[t]$ 可通过现有深度学习框架自动求导计算；
- **直观解释：** 论文通过 FTRL 视角说明 NDOT 以连续时间依赖隐式捕捉历史信息，弥补仅依赖即时损失方法的不足。

Algorithm 1 NDOT 一次迭代

```

1: Input: 训练样本  $(x, y)$ , 时间步数  $T$ 。
2: Initialize  $\hat{a}^l[0] = 0$  和  $e^l[0] = 0$ 。
3: for  $t = 1, \dots, T$  do
4:   for  $l = 1, \dots, N$  do
5:     前向计算膜电位  $u^l[t]$  和脉冲  $s^l[t]$ 。
6:     计算连续时间依赖  $e^l[t] = \frac{u^l[t] - V_{th}s^l[t]}{u^l[t-1] - V_{th}s^l[t-1]}$ 。
7:     递推时间梯度  $\hat{a}^l[t] = e^l[t-1] \odot \hat{a}^l[t-1] + s^l[t]$ 。
8:   end for
9:   for  $l = N, \dots, 1$  do
10:    计算空间梯度  $g_l^u[t] = (\partial L / \partial s^l[t])(\partial s^l[t] / \partial u^l[t])$ 。
11:    计算即时权重梯度  $\nabla_{W^l} L[t] = g_l^u[t](\hat{a}^{l-1}[t])^\top$ 。
12:    使用梯度优化器更新权重  $W^l$ 。
13:   end for
14: end for

```

V. 实验验证

论文在 CIFAR-10、CIFAR-100 和 CIFAR10-DVS 上验证了 NDOT 的性能, 表明该方法在小时间步设置下仍能保持良好精度, 并显著降低训练内存需求。同时, 论文通过 GPU 内存占用对比, 展示了 NDOT 的在线训练可行性。

VI. 结论

NDOT 通过神经元动力学的连续时间依赖和空间/时间梯度解耦, 实现了可在线执行的前向时间 SNN 训练。该方法兼顾了梯度准确性和内存效率, 是面向 neuromorphic 在线训练的有效方案。

参考文献

- [1] H. Jiang, G. D. Masi, H. Xiong, and B. Gu, “NDOT: Neuronal dynamics-based online training for spiking neural networks,” in *Forty-first International Conference on Machine Learning*, 2024. [Online]. Available: <https://openreview.net/forum?id=eIF0QoBSFV>